

TD02 - C Exercises

Exercises from CM-all-2025

Louis Ledoux

2026-07-05

C Exercises

C Exercises

Live pacing: about 8 min

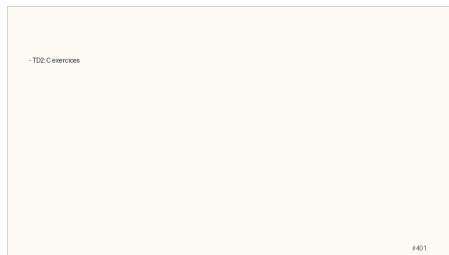
Question. Can students transfer the lecture model to unfamiliar code?

Core claim. Exercises should force a trace: values, addresses, calls, files, or ownership.

Target capability. Students can justify an answer with a state diagram, not only a final value.

Main Path

- ▶ Start from the question: Can students transfer the lecture model to unfamiliar code?
- ▶ Build the model: Exercises should force a trace: values, addresses, calls, files, or ownership.
- ▶ End with a checkable action: Students can justify an answer with a state diagram, not only a final value.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Ask for a prediction first, then require the memory or process trace that supports it.
- ▶ Representative source slide: 402
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

```
- Exercise 1: Basic I/O

- What prints the following program? (assume array nom is stored at the address 0xc34724000)

#include <stdio.h>
int main()
{
    char nom[10] = "Althazar";
    double solde = 1000.25;
    int age = 21;
    printf("Hello %s, you have %d years\n", nom, age);
    printf("Your age is written %x in hex\n", age);
    printf("Your name start with the letter %c\n", nom[0]);
    printf("Your bank account balance is %f\n", solde);
    printf("The address of the table name is %p\n", nom);
}
```

#402

Anecdote Or Motivation

The best exercise answers include a small drawing; C rewards explicit state.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Exercise 7: Files

- Write a main function that
- Reads the name of a file (name1) from the keyboard
- Reads the name of a file (name2) from the keyboard
- Copies character by character the content of the file name1 to the file name2 and replace all the 'X' with 'Y'.
- If the copy is not possible, prints an error message

```
#include <stdio.h>
#include <string.h>
FILE* in;
FILE* out;
char nom1[40];
char nom2[40];
int main()
//To Be Completed
}
```

#410

Backup Index

Original slides 401-410

- TD2: C exercices

Backup: TD2: C exercices

Original slide 401

- TD2: C exercices

Backup: Exercice 1: Basic I/O

Original slide 402

- What prints the following program? (assume array nom is stored at the address 0x3472A000)

```
#include<stdio.h>
int main(){
char nom[20] = "Balthazar";
double solde = 1000.25;
int age = 21;
printf("Hello %s, you have %d years\n", nom, age);
printf("Your age is written %x in hex\n", age);
printf("Your name start with the letter %c\n", nom[0]);
printf("Your bank account balance is %lf\n", solde);
printf("The address of the table name is %p\n", nom);
}
```



Backup: Exercice 2: Pointers

Original slide 403

- ▶ Assume the following instructions `int x=0; int y=0;`
`int* p=NULL; p = &x; x=2; y=4; *p = y; x = 5; y =`
`*p; p = &y;`
- ▶ Provide the values of `x`, `y` and `p`

Exercice 2: Pointers

- Assume the following instructions: `int x=0, int y=0, int* p=NULL, p=&x; x=2; y=4; *p=y;`
`x=5; y=&x; p=&y;`
- Provide the values of `x`, `y` and `p`

#403

Backup: Exercice 3: Pointers

Original slide 404

► Give the output of the program

```
#include <stdio.h>

void exchange(int * a, int b){
    int sauve; sauve = *a; *a = b; b = sauve;
}

void exchange2(int * a, int ** b){
    int sauve; sauve = *a; *a = **b; **b = sauve;
}

void exchange3(int * a, int * b){
    int sauve; sauve = *a; *a = *b; *b = sauve;
}

int main(){
    int a, b, d; int* c = &d; a = 1; b = 2; (*c) = 3;
    exchange(&a,b); printf("a= %d, b= %d\n",a,b);
    exchange2(&a,&c); printf("a= %d, c= %d\n",a,*c);
    exchange3(a,b); printf("a= %d, b= %d\n",a,b);
}
```



Backup: Exercise 4: Strings

Original slide 405

- ▶ Write a function called reverse that reverses the characters of a string, without declaring a second array. The prototype of the function is the following:
- ▶ `int reverse(char* str) ;`

Exercise 4: Strings

- Write a function called reverse that reverses the characters of a string, without declaring a second array. The prototype of the function is the following:
`int reverse(char* str);`

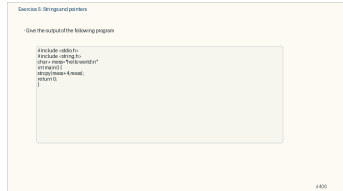
#405

Backup: Exercice 5: Strings and pointers

Original slide 406

► Give the output of the following program

```
#include <stdio.h>
#include <string.h>
char* mess="hello world\n"
int main() {
strcpy(mess+4,mess) ;
return 0;
}
```



Backup: Exercise 6: Linked lists

Original slide 407

- ▶ We have the structure `list_elem_t` for the list elements

```
typedef struct s_list {  
    int value; // data of the element  
    struct s_list* next; // pointer to the next element  
} list_elem_t;
```

- ▶ The head of list is pointing by the pointer `list_head`
- ▶ `list_elem_t* list_head;`
- ▶ Write a function
- ▶ `list_elem_t* create_element(int val),`
- ▶ which creates a new list element and initializes its value equal to the passed argument `val` and the next pointer to NULL

Exercise 6: Linked lists

- We have the structure `list_elem_t` for the list elements

```
typedef struct s_list {  
    int value; // data of the element  
    struct s_list* next; // pointer to the next element  
} list_elem_t;
```

- The head of list is pointing by the pointer `list_head`

```
list_elem_t* list_head;
```

- Write a function
- `list_elem_t* create_element(int val)`
which creates a new list element and initializes its value equal to the passed argument `val`

#407

Backup: Exercice 6: Linked lists

Original slide 408

- ▶ Assume the function `insert_head`, which inserts a new element at the head of the list that has been passed as parameter

```
int insert_head(list_elem_t* l, int val) {  
    list_elem_t * nouveau = create_element(val);  
    nouveau->next = l ;  
    return 0 ;  
}
```

- ▶ Is it correct?
- ▶ What happens when `insert_head(list_head, 1)` is executed?
- ▶ Correct the function !

Source 6: Linked list

- Analyze the function `insert_head`, which inserts a new element at the head of the list that has been passed as parameter

```
int insert_head(list_elem_t* l, int val) {  
    list_elem_t * nouveau = create_element(val);  
    nouveau->next = l;  
    return 0;  
}
```

- Is it correct?
- What happens when `insert_head(list_head, 1)` is executed?
- Correct the function!

#408

Backup: Exercise 6: Linked lists

Original slide 409

Write a function:

```
int insert_tail(list_elem_t** l, int val),  
which creates a new list element at the tail of the
```

Exercise 6: Linked lists

Write a function
int insert_tail(list_elem_t** l, int val),
which creates a new list element at the tail of the list and return 0 in case of success and -1
in case of failure

#409

Backup: Exercise 7: Files

Original slide 410

- ▶ Write a main function that
- ▶ Reads the name of a file (name1) from the keyboard
- ▶ Reads the name of a file (name2) from the keyboard
- ▶ Copies character by character the content of the file name1 to the file name2 and replace all the 'X' with 'Y'.
- ▶ If the copy is not possible, prints an error message

```
#include <stdio.h>
#include <string.h>
FILE* in;
FILE* out;
char nom1[40];
char nom2[40];
int main() {
//To Be Completed
}
```

